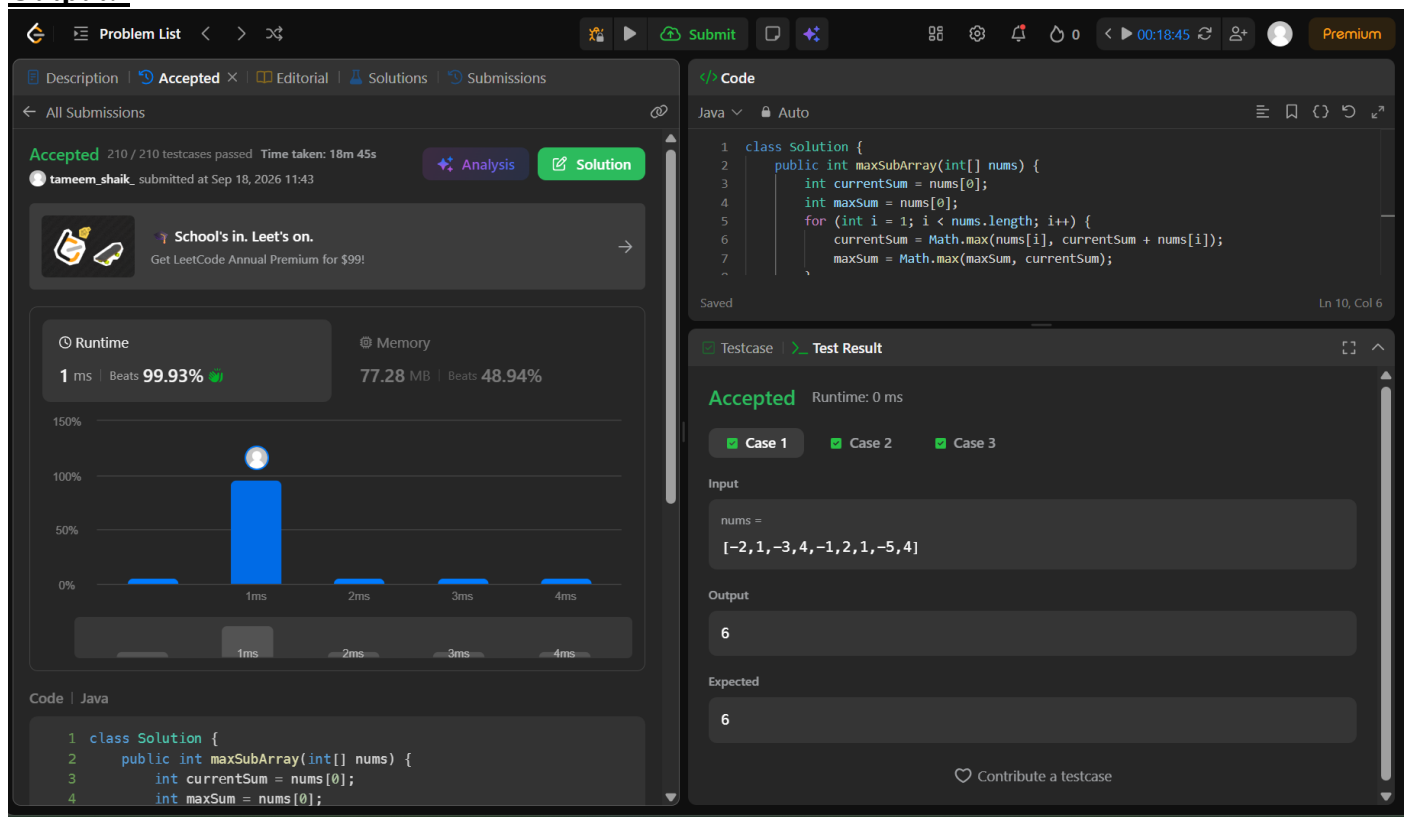


Maximum Subarray:-

Code:-

```
class Solution {  
    public int maxSubArray(int[] nums) {  
        int currentSum = nums[0];  
        int maxSum = nums[0];  
        for (int i = 1; i < nums.length; i++) {  
            currentSum = Math.max(nums[i], currentSum + nums[i]);  
            maxSum = Math.max(maxSum, currentSum);  
        }  
        return maxSum;  
    }  
}
```

Output:-



The screenshot displays the LeetCode interface for the 'Maximum Subarray' problem. The top section shows the problem description and the user's submission status: 'Accepted' with 210/210 testcases passed and a time taken of 18m 45s. The user's name is 'tameem_shaik' and the submission was made on Sep 18, 2026 at 11:43. Below this, there is a promotional banner for 'School's in. Leet's on.' and a 'Solution' button.

The 'Runtime' section shows a bar chart comparing the user's performance to other solutions. The user's runtime is 1 ms, which is 99.93% better than other solutions. The 'Memory' section shows the user's memory usage is 77.28 MB, which is 48.94% better than other solutions.

The 'Code' section shows the user's Java solution:

```
1 class Solution {  
2     public int maxSubArray(int[] nums) {  
3         int currentSum = nums[0];  
4         int maxSum = nums[0];  
5         for (int i = 1; i < nums.length; i++) {  
6             currentSum = Math.max(nums[i], currentSum + nums[i]);  
7             maxSum = Math.max(maxSum, currentSum);  
8         }  
9         return maxSum;  
10    }  
11 }
```

The 'Testcase' section shows the results of three test cases, all of which passed. The input for Case 1 is 'nums = [-2, 1, -3, 4, -1, 2, 1, -5, 4]' and the output is '6'. The expected output is also '6'.

Subarray Division:-

Code:-

```
import java.io.*;
import java.util.*;
import java.util.stream.*;
import static java.util.stream.Collectors.toList;

class Result {

    public static int birthday(List<Integer> s, int d, int m) {
        int count = 0;

        for (int i = 0; i <= s.size() - m; i++) {
            int sum = 0;

            for (int j = i; j < i + m; j++) {
                sum += s.get(j);
            }

            if (sum == d) {
                count++;
            }
        }

        return count;
    }
}

public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader =
            new BufferedReader(new InputStreamReader(System.in));

        BufferedWriter bufferedWriter =
            new BufferedWriter(new FileWriter(System.getenv("OUTPUT_PATH")));

        int n = Integer.parseInt(bufferedReader.readLine().trim());

        List<Integer> s =
            Stream.of(bufferedReader.readLine().trim().split(" "))
                .map(Integer::parseInt)
                .collect(toList());

        String[] firstMultipleInput =
            bufferedReader.readLine().trim().split(" ");

        int d = Integer.parseInt(firstMultipleInput[0]);
        int m = Integer.parseInt(firstMultipleInput[1]);

        int result = Result.birthday(s, d, m);
```

```
bufferedWriter.write(String.valueOf(result));
bufferedWriter.newLine();

bufferedReader.close();
bufferedWriter.close();
}
}
```

Output:-

✓ Test case 0

✓ Test case 1

✓ Test case 2

✓ Test case 3 

✓ Test case 4 

✓ Test case 5 

✓ Test case 6 

Compiler Message

Success

Input (stdin)

1	5
2	1 2 1 3 2
3	3 2

Expected Output

1	2
---	---

Download

Download